

Parte 1 — Reconocimiento facial para inicio de sesión

RP_PF_Login_Facial

Índice

1	Parte 1 - Reconocimiento facial para inicio de sesión	2
1.1	Visión general	2
1.2	Estructura del proyecto y datos	2
1.3	Adquisición y preprocesamiento visual	2
1.3.1	Captura temporal	2
1.3.2	Detección Haar	4
1.3.3	Preprocesamiento	4
1.4	Entrenamiento del clasificador	4
1.4.1	Partición del dataset	4
1.4.2	Arquitectura por transfer learning	4
1.5	Evaluación offline	4
1.6	Inferencia temporal y política de decisión	5
1.6.1	Demostración: login simulado	5
1.7	Integración con el entorno de escritorio	7
1.8	Implementación, pruebas y despliegue	8
1.9	Consideraciones de privacidad y seguridad	8
1.10	Estado actual	8

1 Parte 1 - Reconocimiento facial para inicio de sesión

1.1 Visión general

El componente *RP_PF_Login_Facial* implementa un flujo de **identificación facial 1:N** para proponer el inicio de sesión en KDE Plasma. Reconoce dos identidades entrenadas (**elioth**, **emmanuel**) y asigna **guest** cuando la evidencia no supera los umbrales configurados. **guest** no es una clase del clasificador: es un mecanismo de rechazo en la capa de decisión.

La Figura 1 resume la separación entre entrenamiento offline e inferencia temporal en login. La identificación (visión + ML) se mantiene desacoplada de la autenticación del SO (PAM, greetd).

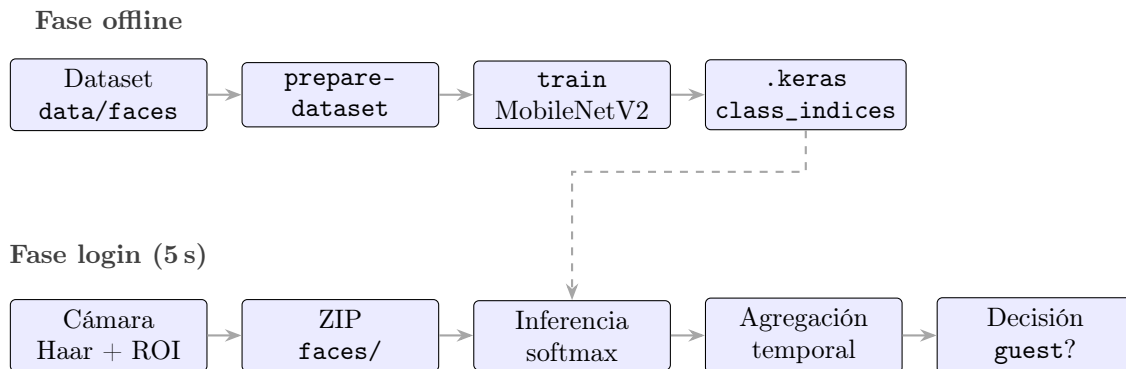


Figura 1: Arquitectura general: entrenamiento offline e inferencia temporal en login.

1.2 Estructura del proyecto y datos

El paquete `src/rp_face_login/` se organiza en seis módulos funcionales más `cli.py` y `config.py`. La configuración central reside en `configs/default.yaml`.

Cuadro 1: Módulos del pipeline facial.

Módulo	Responsabilidad
acquisition/	Captura temporal sin vista previa; ZIP con rostros recortados.
vision/	Detección Haar, recorte con margen, redimensionado.
training/	Dataset <code>train/val/test</code> , entrenamiento y evaluación.
inference/	Predicción softmax por frame y agregación temporal.
decision/	Política de aceptación o rechazo (guest).
session/	Despacho simulado o integración experimental con greetd.

Los datos y artefactos se generan localmente (excluidos de git por privacidad). La estructura esperada se muestra en los Listados 1 y 2.

Tras `prepare-dataset`, cada imagen válida queda como rostro recortado y redimensionado a 224×224 . La Figura 2 ilustra un ejemplo de muestra procesada lista para el clasificador.

1.3 Adquisición y preprocesamiento visual

1.3.1 Captura temporal

El módulo `acquisition/camera_capture.py` abre la cámara (por defecto índice 0, 1280×720 , 5s), lee el máximo de frames posible y, por cada frame con detección, guarda el recorte en un

Listing 1: Estructura de data/.

```
data
|-- faces (caras extraidas sin procesar)
|   |-- elioth
|   |   |-- luz_ambiental
|   |   |-- luz_frontal
|   |-- emmanuel
|   |   |-- luz_ambiental
|   |   |-- luz_frontal
|-- processed (caras preparadas para entrenamiento)
|   |-- test
|   |   |-- elioth
|   |   |-- emmanuel
|   |-- train
|   |   |-- elioth
|   |   |-- emmanuel
|   |-- val
|   |   |-- elioth
|   |   |-- emmanuel
'-- raw (videos mp4 de la webcam)
    |-- elioth
    |-- emmanuel
```

Listing 2: Artefactos en models/.

```
models
|-- class_indices.json
|-- face_auth_model.keras
'-- history.json
```

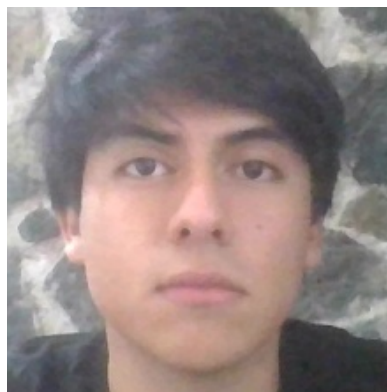


Figura 2: Ejemplo de rostro extraído y recortado (224×224) para entrenamiento o inferencia.

ZIP en memoria (`faces/face_NNNN.jpg`). No se usa `cv2.imshow`.

1.3.2 Detección Haar

OpenCV aplica `haarcascade_frontalface_default.xml` con `scale_factor = 1.1`, `min_neighbors = 6` y tamaño mínimo 100×100 px. Ante varias detecciones se conserva la de mayor área.

1.3.3 Preprocesamiento

El pipeline (`vision/preprocessing.py`) recorta con margen de 30 px, redimensiona a 224×224 , convierte BGR→RGB y mantiene el rango $[0, 255]$ para que `preprocess_input` del backbone normalice según ImageNet.

1.4 Entrenamiento del clasificador

1.4.1 Partición del dataset

`prepare-dataset` divide las imágenes aceptadas en proporciones 70 % / 15 % / 15 % (`train/val/test`) con `seed = 42`. Las estadísticas se guardan en `dataset_stats.json`.

1.4.2 Arquitectura por transfer learning

El entrenamiento (`training/train_model.py`) congela MobileNetV2 (preentrenado en ImageNet) y añade una cabeza softmax de dos clases:

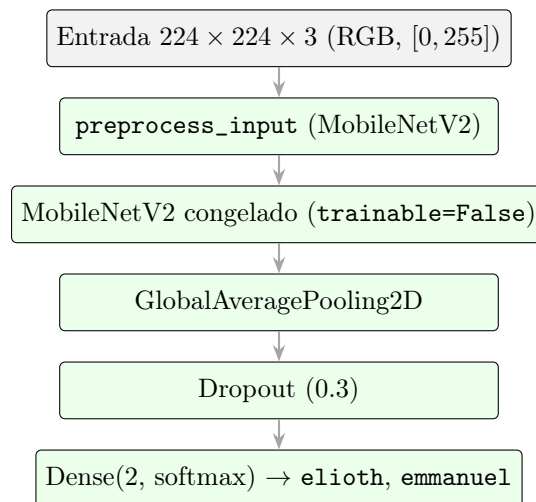


Figura 3: Arquitectura del clasificador facial.

Pérdida: entropía cruzada categórica. Métricas monitorizadas: *accuracy*, *precision*, *recall*. Hipérparámetros por defecto: 10 épocas, *batch size* 32, Adam con $lr = 10^{-3}$.

1.5 Evaluación offline

`evaluate` infiere sobre `data/processed/test/` y genera `reports/evaluation.json` y `reports/confusion_matrix`. La Figura 4 corresponde a una ejecución real del proyecto sobre el conjunto de test local.

Con los conteos de la matriz, la precisión global en test es $accuracy = (175+162)/337 = 1,00$. Estos resultados dependen del dataset y condiciones de captura locales; no se versionan en el repositorio.

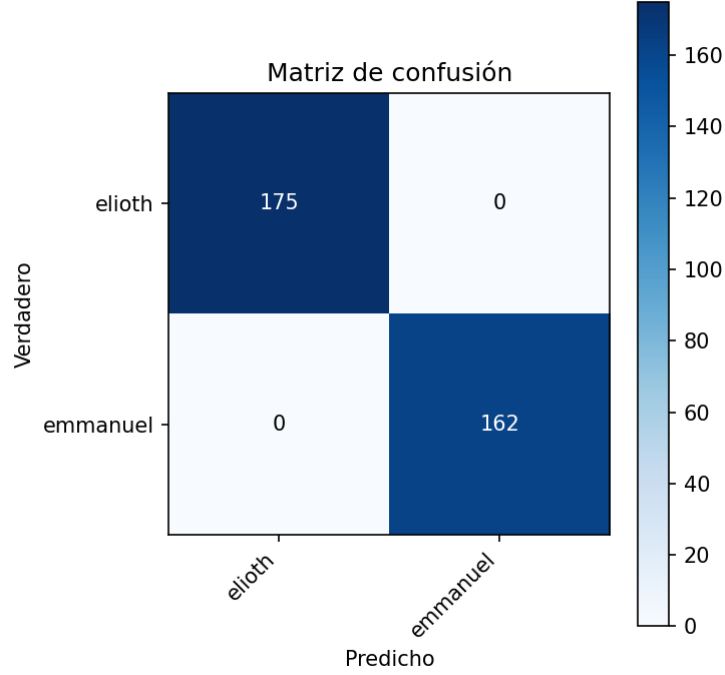


Figura 4: Matriz de confusión en el conjunto de test ($N = 337$ muestras: 175 `elioth`, 162 `emmanuel`). No se observan confusiones entre clases.

1.6 Inferencia temporal y política de decisión

En login el modelo **no se reentrena**. `predict-zip` produce, por cada frame i , probabilidades $P(c_j | \text{frame}_i)$. La agregación temporal promedia por clase:

$$\bar{s}(c_j) = \frac{1}{N} \sum_{i=1}^N P(c_j | \text{frame}_i) \quad (1)$$

Se obtienen `best_user`, `second_user` y `margin = best_score - second_score`. La aceptación exige las tres condiciones de la Tabla 2.

Cuadro 2: Umbrales por defecto (`configs/default.yaml`).

Parámetro	Valor	Condición
<code>min_valid_frames</code>	30	<code>valid_frames</code> $\geq$ 30
<code>confidence_threshold</code>	0.80	<code>best_score</code> $\geq$ 0.80
<code>margin_threshold</code>	0.25	<code>margin</code> $\geq$ 0.25

Si falla alguna condición, `selected_user` pasa a `guest` con motivos `insufficient_frames`, `low_confidence` o `margin_below_threshold`.

1.6.1 Demostración: login simulado

La Figura 5 muestra la salida de `login-sim` en un caso donde el clasificador identifica con alta confianza a `elioth` (`best_score` $\approx$ 0,997, `margin` $\approx$ 0,994), pero la política rechaza el acceso por `insufficient_frames` (22 frames válidos < 30). Esto ilustra que **alta confianza softmax no implica aceptación** si no se cumplen todos los umbrales.

```
[aliiothosa@archlioth PF_PAT]$ .venv/bin/python -m rp_face_login.cli login-sim --model models/face_auth_model.keras --output
-dir ./capturas --save-decision reports/decision.json --duration 5
Captura finalizada.
Frames leídos : 44
Frames válidos : 22
ZIP generado : capturas/login_20260611_225757.zip
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1781240282.606339 26204 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
I0000 00:00:1781240282.909825 26204 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instru
ctions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1781240286.931186 26204 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
E0000 00:00:1781240290.603054 26204 cuda_executor.cc:1737] INTERNAL: CUDA Runtime error: Failed call to cudaGetRuntimeVersion
: Error loading CUDA libraries. GPU will not be used.: Error loading CUDA libraries. GPU will not be used.
W0000 00:00:1781240290.603554 26357 cuda_executor.cc:1755] Failed to determine cuDNN version (Note that this is expected if t
he application doesn't link the cuDNN plugin): INTERNAL: cuDNN error: CUDNN_STATUS_INTERNAL_ERROR
W0000 00:00:1781240290.627915 26204 gpu_device.cc:2365] Cannot dlopen some GPU libraries. Please make sure the missing librar
ies mentioned above are installed properly if you would like to use GPU. Follow the guide at https://www.tensorflow.org/install
/gpu for how to download and setup the required libraries for your platform.
Skipping registering GPU devices...
Inferencia finalizada.
Frames predichos : 22

=====
LOGIN FACIAL (SIMULACIÓN)
=====
Usuario seleccionado : guest
Aceptado : False
Motivo : insufficient_frames
Frames válidos : 22
Mejor : elioth (0.997)
Segundo : emmanuel (0.003)
Margen : 0.994
=====
(Demostración: no se inicia ninguna sesión real.)
decisión guardada : reports/decision.json
[aliiothosa@archlioth PF_PAT]$
```

Figura 5: Salida de login-sim: rechazo a guest pese a alta confianza en elioth (reason = insufficient_frames).

El Listado 3 reproduce el `decision.json` asociado a esa ejecución. El Listado 4 muestra las primeras predicciones por frame del mismo lote.

Listing 3: Decisión de login (`decision_example.json`).

```
{
  "selected_user": "guest",
  "accepted": false,
  "reason": "insufficient_frames",
  "valid_frames": 22,
  "best_user": "eliath",
  "best_score": 0.996796502308412,
  "second_user": "emmanuel",
  "second_score":
    0.0032034812828979657,
  "margin": 0.9935930210255141,
  "thresholds": {
    "min_valid_frames": 30,
    "confidence_threshold": 0.8,
    "margin_threshold": 0.25
  }
}
```

Listing 4: Primeras predicciones por frame (`predictions_example.json`).

```
[
  {
    "frame": "face_0001.jpg",
    "eliath": 0.9964351654052734,
    "emmanuel": 0.003564809914678335
  },
  {
    "frame": "face_0002.jpg",
    "eliath": 0.9984868764877319,
    "emmanuel": 0.0015130412066355348
  },
  {
    "frame": "face_0003.jpg",
    "eliath": 0.9995261430740356,
    "emmanuel": 0.0004739107971545309
  },
  {
    "frame": "face_0004.jpg",
    "eliath": 0.9993948936462402,
    "emmanuel": 0.0006051086820662022
  },
  {
    "frame": "face_0005.jpg",
    "eliath": 0.999718427658081,
    "emmanuel": 0.0002815413463395089
  },
  {

```

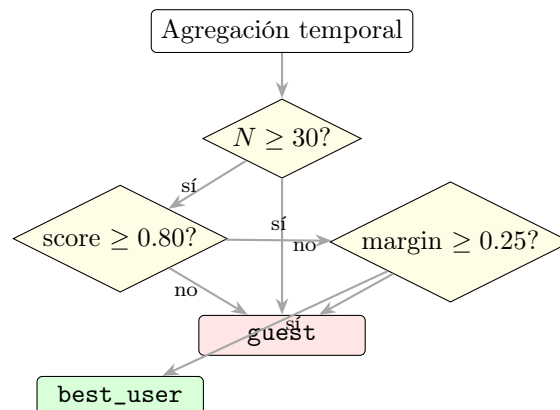


Figura 6: Diagrama de la política de decisión (las tres condiciones deben cumplirse).

1.7 Integración con el entorno de escritorio

El greeter experimental (`scripts/face-login-greeter.sh`) encadena `login-sim` con el despachador de sesión. La Figura 7 muestra el arranque bajo `greetd` en modo `greetd-ipc`, con rutas de despliegue en `/opt/rp_face_login` y ejecución del paso 1/3 (`login-sim`).

Se distinguen tres niveles documentados en `docs/deploy_endeavours_vm.md`:

- **L1: dry-run** — valida ML sin abrir Plasma.

```
[2026-06-11 23:13:40] === face-login-greeter ===
[2026-06-11 23:13:40] repo=/opt/rp_face_login config=/opt/rp_face_login/configs/default.yaml model=/opt/rp_face_login/models/face_auth_model.keras log=/opt/rp_f
ace_login/logs/face-login.log
[2026-06-11 23:13:40] login=/home/elieth/RP_PF_Login_Facial/.venv/bin/python dispatch_mode=greetd-ipc
[2026-06-11 23:13:40] Paso 1/3: ejecutando login-sim...
```

Figura 7: Log de face-login-greeter en VM: configuración, modelo y dispatch_mode=greetd-ipc.

- **L2:** demo híbrida con greeter gráfico y contraseña PAM.
- **L3:** greetd-ipc — PAM + startplasma-wayland tras identificación facial.

En todos los casos el rostro propone el *username*; PAM sigue siendo la autoridad de autenticación.

1.8 Implementación, pruebas y despliegue

Cuadro 3: Comandos principales del pipeline facial.

Comando	Función
prepare-dataset	Genera data/processed/{train,val,test}.
train	Entrena y guarda face_auth_model.keras.
evaluate	Métricas y matriz de confusión en test.
capture	ZIP temporal de rostros (sin vista previa).
predict-zip	Inferencia softmax por frame desde ZIP.
login-sim	Flujo completo de login simulado.

El entorno se gestiona con uv; dependencias ML en el extra [ml] (TensorFlow, scikit-learn, Matplotlib). La suite tests/ cubre detección, preprocesamiento, entrenamiento, inferencia, agregación, decisión y despacho. Opcionalmente, PyInstaller genera face-login-capture y face-login-sim.

1.9 Consideraciones de privacidad y seguridad

- Rostros, ZIP de captura, modelos y reportes son datos sensibles (.gitignore); las figuras de este documento provienen de ejecuciones locales controladas.
- El softmax puede ser confiado ante rostros no vistos; los umbrales y guest mitigan falsos positivos.
- Un lote de login **nunca** debe usarse para reentrenar el modelo.
- En integración real no se debe omitir PAM ni hardcodear contraseñas.

1.10 Estado actual

El pipeline facial está implementado de extremo a extremo: preparación de dataset, entrenamiento, evaluación offline, captura sin vista previa, inferencia con agregación temporal y política de rechazo explícita. Las Figuras 4–5 documentan resultados reales del entorno de desarrollo. La integración con greetd (Figura 7) se valida en VM como fase experimental posterior al login-sim.